ASCEND SPORTS PLATFORM

Full-Stack Sports Intelligence, CrickHeroes Manual Scoring & 3D WebGL Arena

React 18 • TypeScript 5 • Vite • Tailwind CSS • Three.js • Firebase Firestore • PWA Workbox

Official Codebase Download Archive: /ascend-sports-platform.zip (324 KB)

SYSTEM ARCHITECTURE & CORE SUBSYSTEMS

Engineered with modular React architecture and Firestore synchronization

1. CrickHeroes Manual Cricket Scoring Engine
Ball-by-ball real-time scoring, legal deliveries, striker auto-rotation, bowler over restrictions, 6 dismissal modes, and instant undo snapshots.

2. Dynamic Milestone Calculation Rules
Automated 50s and 100s threshold engine: Short format (<=12 overs) and Test (<=45 overs) evaluate 30-59 runs as 50 and 60+ as 100.

3. CrickHeroes Football Scorer & Analytics
Configurable half durations (10-45m), automated half-time clock transitions, pitch-position goal logging, GK save tracker (+50 pts), and defensive masterclass logger

4. Playable 3D WebGL Game Arena (Cricket & Football)
Three.js browser game with national & associate rosters (India, Australia, Japan, Nepal), bilateral series, World Cup tournaments, and configurable match lengths.

5. Real-Time Multi-Language Commentary Engine
Simultaneous 3-language commentary generation (English, Hindi, Bengali) with contextual match situations, ball speeds, and periodic Sidhuisms.

6. Guddu Multi-Sport Rules & Referee Chatbot
Comprehensive knowledge base covering Cricket, Football, Kabaddi, Tennis, Volleyball, Rugby, Polo, Golf, and Racquet sports with offline search and voice synthes

7. Player Ability Testing & Radar Chart Module
100m sprint stopwatch, shot velocity radar, tackle success drills, and bowling speed logging with automatic Firestore persistence and public radar cards.

Key Build & Environment Specifications:

- Framework: React 18.3 (Single Page Application via Vite 6.2)
- Language: TypeScript 5.7 (Strict mode, 0 lint/compile errors)
- Database & Auth: Firebase Firestore & Firebase Authentication (ai-studio-992e55ae-29b1-492b-86b5-9a1b40d169b5)
- 3D Engine: Three.js r128 with WebGL hardware acceleration, camera controls, dynamic lighting, and field textures
- Offline Support: Vite PWA Plugin + Workbox caching (6MB cache threshold for Three.js assets and offline rule database)
- Export Utilities: Client-side jsPDF match scorecard printing, CSV spreadsheet export, JSON match payloads, and ZIP archive packaging

COMPLETE CODEBASE FILE MANIFEST (71 FILES)

Cleanly separated component architecture following React & TypeScript best practices

Root Configuration & Build Files

- package.json (Dependencies, scripts, Vite, Three.js, jsPDF, Lucide icons)
- vite.config.ts (Vite build bundler, PWA plugin, Workbox 6MB cache, path alias)
- tsconfig.json & tsconfig.node.json (TypeScript strict compilation settings)
- index.html (HTML5 entry point, metadata, manifest link, favicon)
- firestore.rules (Firestore security rules with athlete RBAC and audit logs)
- metadata.json (AI Studio applet configuration and capabilities)
- .env.example (Environment variable templates)

Application Core & Routing (/src)

- src/App.tsx (Main application container, tab switcher, Guddu dock, PWA notification)
- src/main.tsx (React DOM root mount)
- src/index.css (Tailwind CSS global styling, animations, scrollbars)
- src/types/models.ts (Comprehensive TypeScript interfaces for sports, stats, matches, users, commentary)

Core Business Logic & Engines (/src/lib)

- src/lib/milestoneEngine.ts (Dynamic 50s/100s rule calculation for short & standard matches)
- src/lib/commentaryEngine.ts (Multi-language commentary engine in EN, HI, BN + Sidhuisms)
- src/lib/sportsRulesData.ts (Universal referee knowledge base for 10+ sports)
- src/lib/exportUtils.ts (PDF official match scorecard generator, JSON, CSV, and code zip utilities)
- src/lib/rankingEngine.ts (Global athlete ranking points algorithm for Cricket & Football)
- src/lib/tournamentEngine.ts (Knockout & round-robin bracket fixture generators)
- src/lib/geo.ts (Privacy coordinates fuzzing, geohash calculation, Indian cities database)
- src/lib/firebase.ts (Firestore initialization, auth state hooks, offline cache persistence)

Scoring & Live Match Components (/src/components)

- src/components/CrickHeroesCricketScoringDashboard.tsx (Manual cricket scorer, ball-by-ball actions, undo)
- src/components/CrickHeroesFootballScoringDashboard.tsx (Manual football scorer, pitch-position goals, GK saves)
- src/components/CricbuzzLiveScorecard.tsx (International Cricbuzz-style scorecard with FOW timeline)
- src/components/LiveMatchCenter.tsx (Dual view switcher: Cricbuzz Live View vs CrickHeroes Scorer)
- src/components/MultiLanguageCommentaryFeed.tsx (Live multi-lingual feed with audio speech synthesis)
- src/components/MatchDownloadModal.tsx (Scorecard export modal: PDF, CSV, JSON, Print)

Athlete Abilities, Profiles & 3D Arena (/src/components)

- src/components/PlayerAbilityTestModal.tsx (Standardized skill test drills: speed, attack, defense, radar)
- src/components/ProfileView.tsx (Athlete profile, verified skill radar card, custom stats display)
- src/components/PlayerCard.tsx (Real-time player card with batting/bowling/physical metrics & Test button)
- src/components/GudduChatbot.tsx (Referee AI assistant with quick prompts, rule search, and text-to-speech)
- src/components/PWAInstallNotification.tsx (PWA add-to-home-screen install prompt and guided setup)
- src/components/arena/GameArena.tsx (Interactive Three.js playable cricket and football game engine)

ENGINE IMPLEMENTATION DEEP DIVE

Key algorithms powering ASCEND sports operations

```
// src/lib/milestoneEngine.ts - Dynamic Milestone Evaluation Rules

export function evaluateBattingMilestone(runs: number, totalInningsOvers: number, format: CricketFormat) {
  const isTest = format === "test";
  const isShortOrThresholdMatch = isTest ? totalInningsOvers <= 45 : totalInningsOvers <= 12;

  if (isShortOrThresholdMatch) {
    if (runs >= 60) return { milestoneTitle: "Century (100) [60+ in Short Format]", isCentury: true };
    if (runs >= 30) return { milestoneTitle: "Half-Century (50) [30-59 in Short Format]", isFifty: true };
  } else {
    if (runs >= 100) return { milestoneTitle: "Century (100+ Runs)", isCentury: true };
    if (runs >= 50) return { milestoneTitle: "Half-Century (50-99 Runs)", isFifty: true };
  }
  return { isFifty: false, isCentury: false };
}
```

```
// src/lib/commentaryEngine.ts - Multi-Lingual Generator & Sidhuisms

export function generateCricketCommentary(matchId, overStr, ballNumber, eventType, wicketType, bowler, striker) {
  const isEvery4th = ballNumber > 0 && ballNumber % 4 === 0;
  let textSet = isEvery4th ? SIDHUISMS[Math.floor(Math.random() * SIDHUISMS.length)] : CRICKET_LINES[eventType];
  return {
    matchId, sport: "cricket", periodOrOver: `Over ${overStr}`, ballOrMinute: ballNumber,
    textEn: `${striker} - ${textSet.en}`,
    textHi: `${striker} - ${textSet.hi}`,
    textBn: `${striker} - ${textSet.bn}`,
    timestamp: Date.now()
  };
}
```

QUICK START & LOCAL INSTALLATION INSTRUCTIONS

1. Extract the downloaded ZIP archive:
   unzip ascend-sports-platform.zip -d ascend-app && cd ascend-app

2. Install all dependencies:
   npm install (or bun install)

3. Start the local development server:
   npm run dev
   Navigate to http://localhost:3000 to launch the application.

4. Production Build:
   npm run build (Outputs optimized bundles with PWA service worker into /dist)

5. Verification & Testing:
   npm run lint   (Runs TypeScript typecheck: tsc --noEmit)